

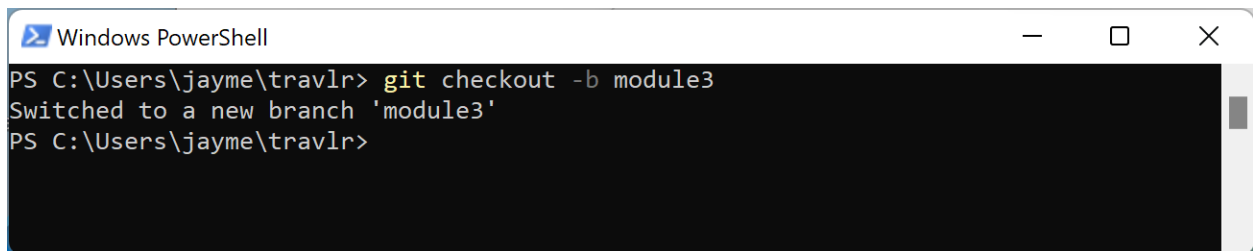
Module 3: Static HTML to Templates with JSON

In this module, we are going to replace static HTML with templates that will utilize JSON to format and display information.

Create Git Branch for Module 3

Before you begin, it is important to make sure that you have created your new branch in git for Module 3. To accomplish this, we will perform the following command in a PowerShell window in the travlr project directory:

```
git checkout -b module3
```



```
Windows PowerShell
PS C:\Users\jayne\travlr> git checkout -b module3
Switched to a new branch 'module3'
PS C:\Users\jayne\travlr>
```

Replacing Static HTML with Templates

We are going to be working on the Trips displayed on the Travel page on the website. We have already started to update the raw travel.html page by turning it into a rendered page using handlebars. We have separated out the header and footer into partials and reduced the code in the remaining template. Now we are going to work on the content of the trips displayed on the page.

To accomplish this, we are going to replace the hard-coded HTML trip content with a loop that will render JSON data using handlebars directives.

1. This is the display of one of the trips currently embedded as static HTML on the page:

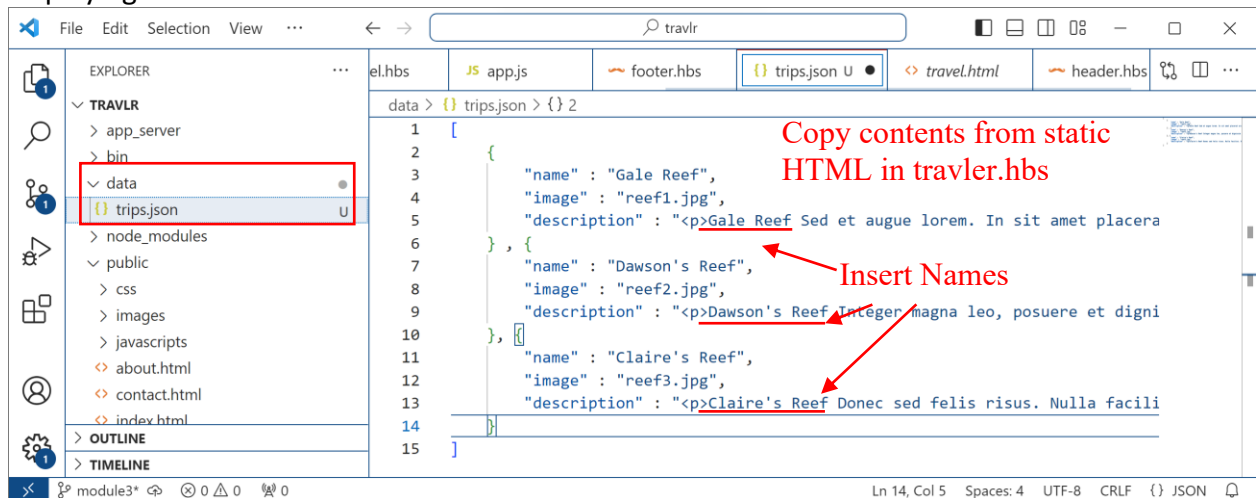
GALE REEF

Sed et augue lorem. In sit amet placerat arcu. Mauris
volutpat ipsum ac justo mollis vel vestibulum orci gravida.
Vestibulum sit amet porttitor odio. Nulla facilisi. Fusce at
pretium felis.

Sed consequat libero ut turpis venenatis ut aliquam risus
semper. Etiam convallis mi vel risus pretium sodales. Etiam
nunc lorem ullamcorper vitae laoreet.



- To enable the transition to a handlebars loop, we will first begin by creating a **data** folder in the main travlr project directory. In the new data folder, we will create a **trips.json** file that will contain a JSON description of the trips for the purposes of testing. **Please Note:** We are asking you to add the name of the trip to the description line in each trip instance so that it is evident when the JSON data is being rendered instead of just displaying the static HTML.

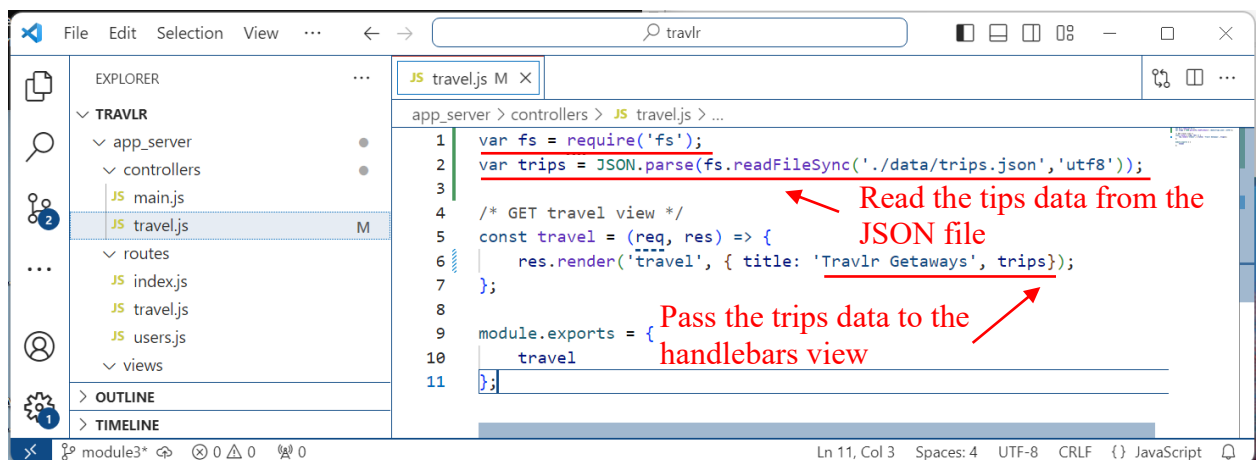


- The next step will be to edit the **travel.js** controller file in order to use the built-in NodeJS filesystem component to read the data file that we just created. We will be using the `fs.readFileSync()` method to retrieve the JSON.

```

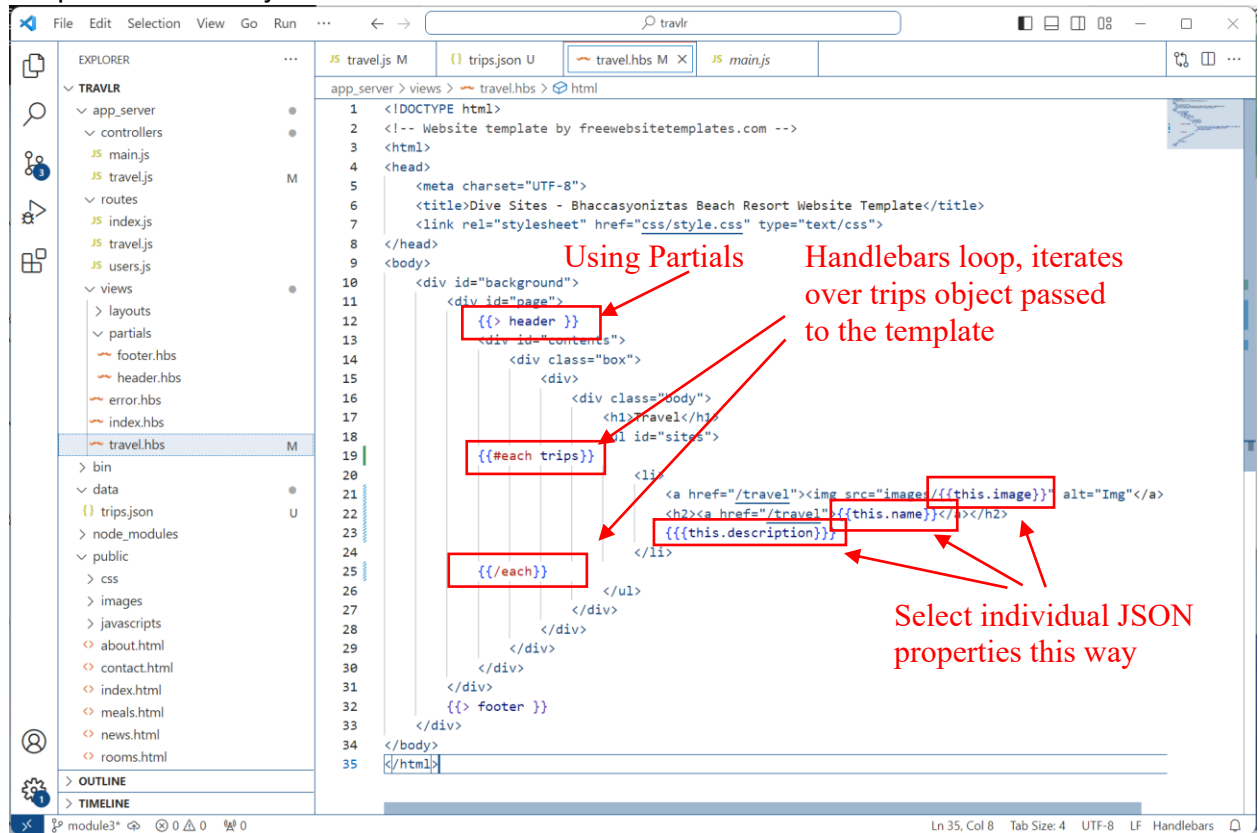
var fs = require('fs');
var trips = JSON.parse(fs.readFileSync('./data/trips.json',
'utf8'));

```

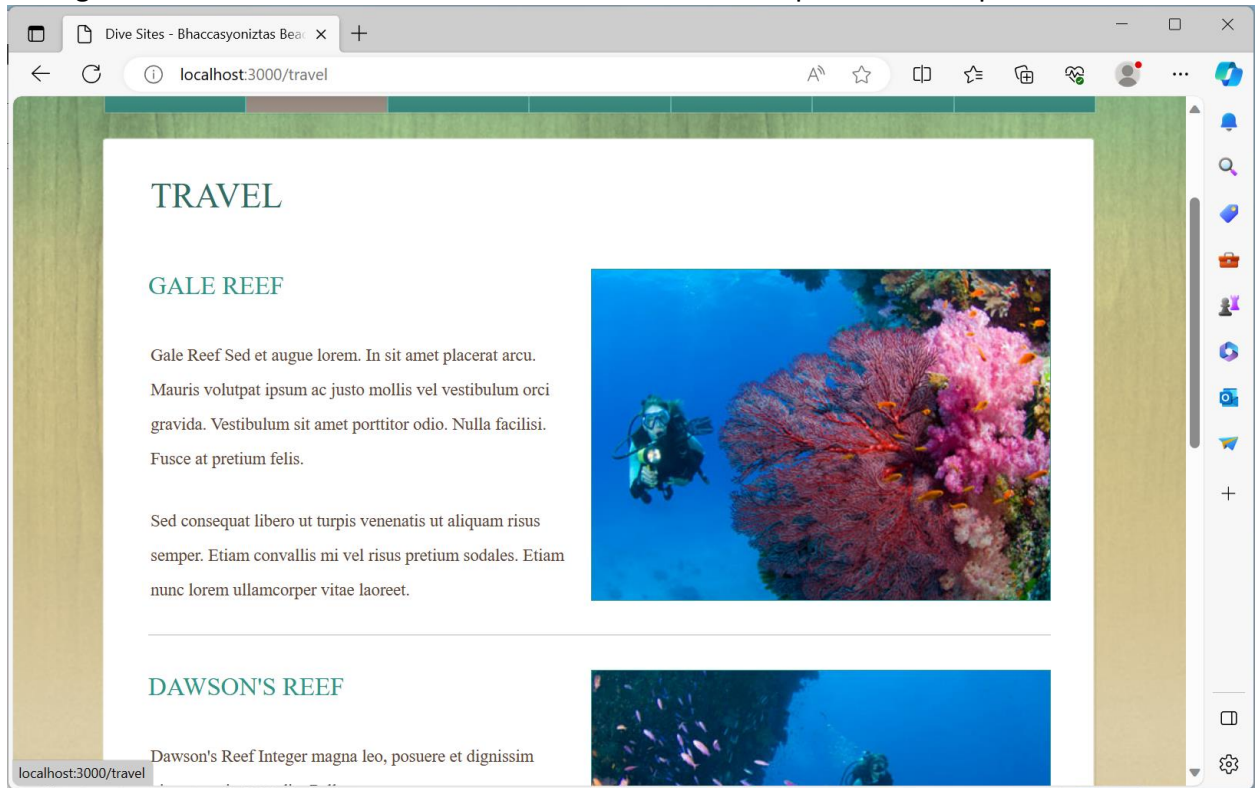


Please Note: It is **not** a best practice to read a JSON file every time the webserver processes a request. This is a method used during development to support rapid prototyping and should be replaced before the applications goes into production.

4. The final step in this process is to edit the `travel.hbs` template and replace the static HTML list entries for the trips with a `{{#each trips}} {{/each}}` directive. This is handlebars notation to create a loop for each object in the 'trips' data collection, allowing you to process each object in a consistent manner.



At this point, you can restart your webserver, and test to make sure that you are now seeing the rendered code which will have the name of the trip in the description as well.

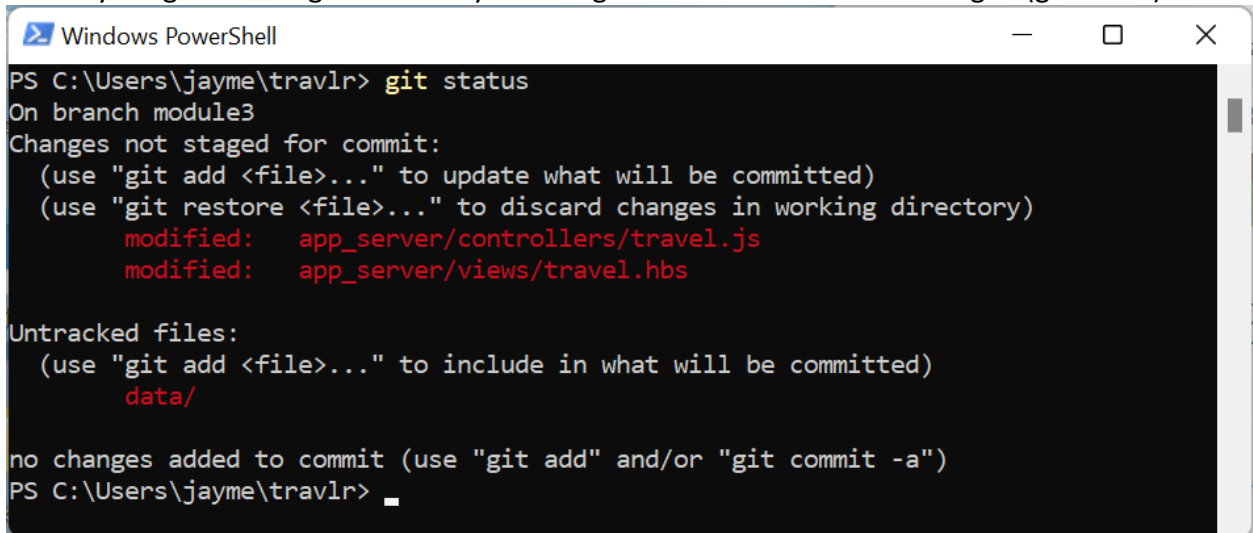


Your effort has replaced 120 lines of static HTML with 35 lines of code including 3 different handlebars directives that allow the rendered page to be driven dynamically with data passed in to the template.

Optional Challenge: Using the techniques that you have just learned, repeat these steps to convert other static HTML pages to Handlebars templates, either with or without JSON data. This is your opportunity to experiment!

Finalizing Module 3

1. Now that we have completed Module 3, we go back to git and make sure that we add everything to tracking. We start by checking the status of what has changed (git status):

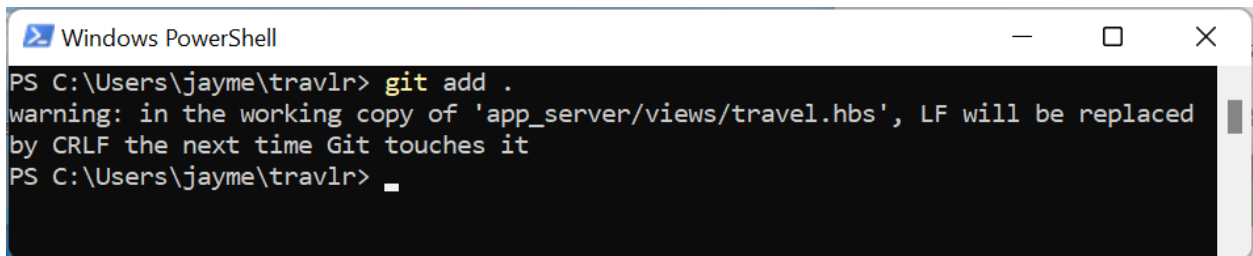


```
Windows PowerShell
PS C:\Users\jayme\travlr> git status
On branch module3
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app_server/controllers/travel.js
        modified:   app_server/views/travel.hbs

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        data/

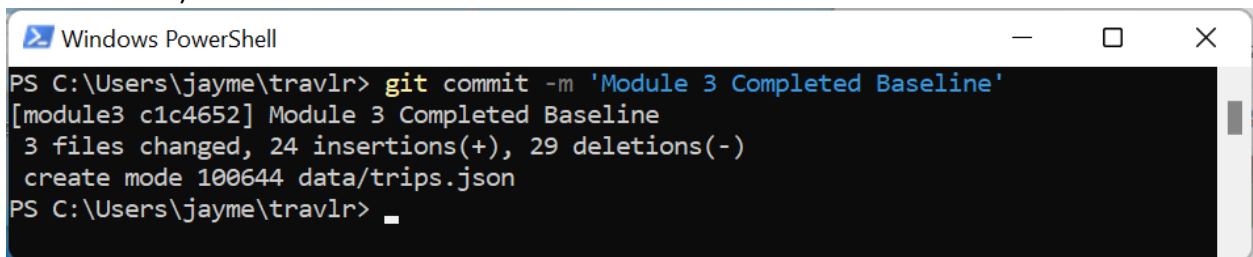
no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\jayme\travlr> _
```

2. Then we add all of those changes into tracking (git add .):



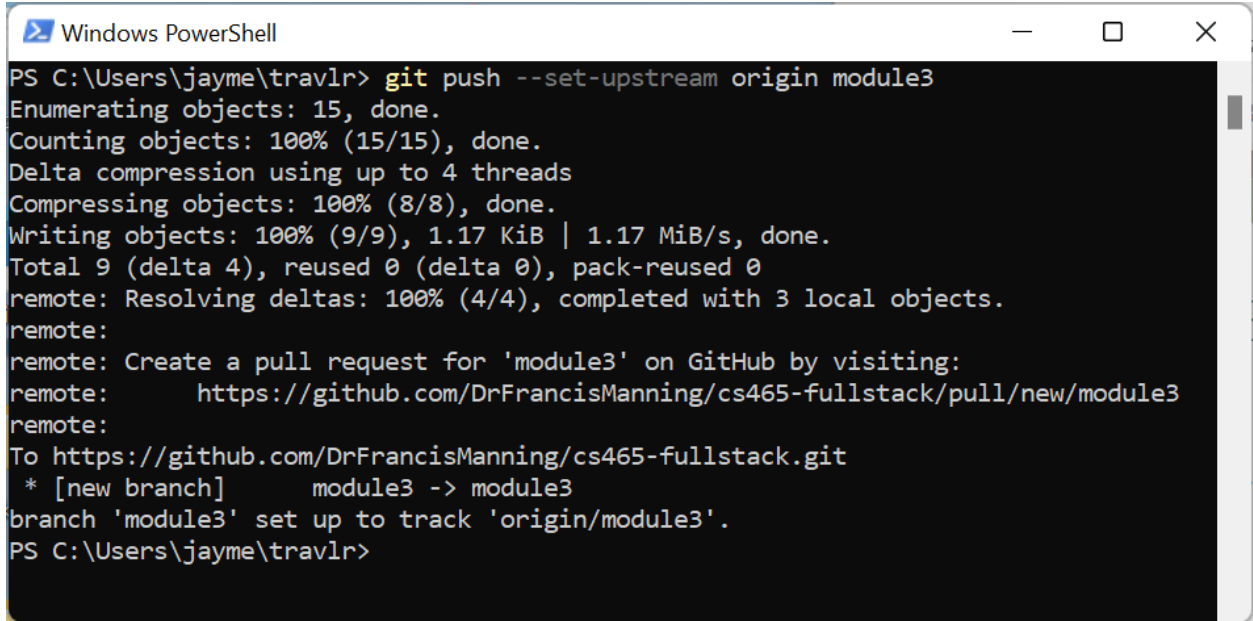
```
Windows PowerShell
PS C:\Users\jayme\travlr> git add .
warning: in the working copy of 'app_server/views/travel.hbs', LF will be replaced
by CRLF the next time Git touches it
PS C:\Users\jayme\travlr> _
```

3. Now we commit the changes (git commit -m 'Module 3 completed baseline'):



```
Windows PowerShell
PS C:\Users\jayme\travlr> git commit -m 'Module 3 Completed Baseline'
[module3 c1c4652] Module 3 Completed Baseline
 3 files changed, 24 insertions(+), 29 deletions(-)
 create mode 100644 data/trips.json
PS C:\Users\jayme\travlr> _
```

4. We push the changes back to GitHub for safekeeping (git push --set-upstream origin module3):



```
Windows PowerShell
PS C:\Users\jayne\travlr> git push --set-upstream origin module3
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 4 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (9/9), 1.17 KiB | 1.17 MiB/s, done.
Total 9 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 3 local objects.
remote:
remote: Create a pull request for 'module3' on GitHub by visiting:
remote:   https://github.com/DrFrancisManning/cs465-fullstack/pull/new/module3
remote:
To https://github.com/DrFrancisManning/cs465-fullstack.git
 * [new branch]      module3 -> module3
branch 'module3' set up to track 'origin/module3'.
PS C:\Users\jayne\travlr>
```